

Tutorial para preparação do ambiente de trabalho

Para trabalhar com Java de forma confortável, você precisa de quatro blocos: sistema operacional atualizado, JDK instalado, MS-VSCode com extensões corretas e uma forma de validar que tudo está realmente funcionando. Para o trabalho em equipe, também recomendo GIT e um terminal bem configurado, porque isso facilita revisão de código, clonagem de repositórios e execução de builds.

1. O que instalar

Base obrigatória

- **JDK 25:** escolha entre uma distribuição open source compatível com Java 25 ou o **Oracle JDK 25**.
- **VS Code:** editor principal de trabalho.
- **Extensões Java para VS Code:** pacote de suporte à linguagem, execução e depuração.
- **Git:** versionamento e integração com repositórios da empresa.

Recomendado para rotina

- **Maven** ou **Gradle**, se os projetos da empresa usarem um desses gerenciadores.
- **Terminal moderno** do próprio sistema ou um terminal adicional.
- **SDKMAN!** no Linux, se a equipe quiser alternar versões de Java com facilidade.
- **WSL 2** no Windows, se a equipe preferir um fluxo mais próximo do Linux para builds e automação.

2. Sistema Operacional: Linux (baseado em Debian)

2.1. Atualizar o sistema

Antes de iniciar a instalação dos requisitos para desenvolvimento, realize a atualização do sistema operacional para reduzir quaisquer problemas de dependências ou versões que possam atrapalhar no trabalho:

```
$ sudo apt update
```

```
$ sudo apt upgrade -y
```

2.2. Instalar o Java

O Java Development Toolkit pode ser instalado e gerenciado de diferentes formas no sistema. Dentre elas, temos a instalação através do Java Oracle, opções de código livre (open source) e/ou gerenciamento de pacotes através do SDKMAN!.

A versão mais recente do Java com suporte de longo prazo (LTS) da plataforma Java SE é a versão 25. No endereço a seguir estão disponíveis os arquivos para as plataformas Linux, MacOS e Windows.

<https://www.oracle.com/br/java/technologies/downloads/#java25>

Se desejar uma distribuição com código-fonte livre (open source) recomendo a leitura do whichJDK e utilizar a distribuição Eclipse Temurin JDK 25. O material está disponível em:

<https://whichjdk.com>

<https://adoptium.net/pt-BR/temurin/releases>

Em ambos os casos, o objetivo é ter `java` e `javac` disponíveis no `PATH`.

Já o SDKMAN! é um gerenciador de versões de Software Development Kits (SDKs) voltado principalmente para desenvolvedores. Ele permite instalar, atualizar e alternar facilmente entre múltiplas versões de linguagens como Java, Kotlin e Scala, além de ferramentas como Maven e Gradle, diretamente pelo terminal, sem alterar variáveis de ambiente manualmente. O website oficial é:

<https://sdkman.io>

Após a instalação através do método escolhido, a verificação no sistema pode ser feita através dos comandos:

```
java -version
```

```
javac -version
```

que devem responder com a versão escolhida.

2.3. Instalar Git

Utilize no shell BASH os comandos a seguir:

```
sudo apt install git -y
```

```
git --version
```

Depois, configure nome e e-mail corporativos:

```
git config --global user.name "Seu Nome"
```

```
git config --global user.email "seuemail@empresa.com"
```

Isso é importante para entrega de tarefas e histórico de autoria.

2.4. Instalar VSCode

No Debian, instale o VSCode pelo pacote oficial da Microsoft através do endereço:

<https://code.visualstudio.com/download>

Depois, confirme que o comando `code` abre o editor e que a interface inicia sem erro.

Extensões essenciais

No VS Code, instale pelo menos:

- **Extension Pack for Java.**
- **Debugger for Java.**
- **Maven for Java**, se a empresa usa Maven.
- **Gradle for Java**, se a empresa usa Gradle.
- **Lombok Annotations Support for VS Code**, se houver Lombok no código.
- **GitLens** ou equivalente, se o time usa inspeção avançada de Git.

2.5. Verificação e teste da instalação

No terminal, confirme:

```
$ java -version
$ javac -version
$ git --version
$ code --version
```

Depois, crie um arquivo simples `HelloWorld.java`, compile e execute:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}

$ javac HelloWorld.java

$ java HelloWorld
Hello, World!

$ _
```

Se isto funcionar, o ambiente está pronto para uso diário.

3. Sistema Operacional: Windows 11

3.1. Atualizar o sistema

Garanta que o Windows Update esteja concluído e reinicie se necessário. Em ambiente corporativo, também vale verificar se há políticas da empresa para instalação de software e execução de scripts.

3.2. Instalar o Java

No Windows 11, você pode seguir dois caminhos:

- **Open source:** instalar uma distribuição compatível com Java 25.

- **Oracle JDK 25:** baixar o instalador oficial da Oracle e seguir o assistente.

Depois da instalação, verifique se a variável de ambiente `JAVA_HOME` foi criada ou ajuste manualmente para apontar para a pasta do JDK. Também confirme que o diretório `bin` do JDK está no `Path`, porque isso permite rodar `java` e `javac` no terminal.

3.3. Instalar Git

Utilize o powershell para instalar o Git para Windows e valide com:

```
git --version
```

Depois, configure nome e e-mail corporativos:

```
git config --global user.name "Seu Nome"
```

```
git config --global user.email "seuemail@empresa.com"
```

Isso é importante para entrega de tarefas e histórico de autoria.

3.4. Instalar VSCode

Baixe e instale o VS Code para Windows 11 e, ao abrir, confirme que o aplicativo inicia corretamente e aceita instalação de extensões. Se a empresa preferir um fluxo mais próximo de Linux, você pode habilitar o WSL 2 e trabalhar com o VS Code conectado ao ambiente Linux local.

<https://code.visualstudio.com/download>

Para Windows, utilize a versão "System Installer" para disponibilizar para todos os usuários do computador.

Extensões essenciais

Use as mesmas extensões do Linux:

- **Extension Pack for Java.**
- **Debugger for Java.**
- **Maven for Java.**
- **Gradle for Java.**
- **Lombok Annotations Support for VS Code.**
- **GitLens.**

3.5. Verificação e teste da instalação

Abra o **PowerShell** ou o terminal do VSCode e execute:

```
$ java -version
$ javac -version
$ git --version
$ code --version
```

Depois, crie um arquivo simples HelloWorld.java, compile e execute:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

```
$ javac HelloWorld.java
```

```
$ java HelloWorld  
Hello, World!
```

```
$ _
```

Se isto funcionar, o ambiente está pronto para uso diário.

4. Ajustes finais

Após a verificação do funcionamento adequado dos programas, versões disponíveis e teste de compilação de um arquivo simples, é importante verificar a depuração com breakpoint no VSCode para confirmar que todo o ciclo de requisitos estão integrados e funcionando corretamente.